

Islamic University of Technology

CSE 4510: Software Development Lab

Project Report

CityZen

A Civic Complaint Management System for Smart Cities

Team Members

1. Ridita Alam	220041110
2. Rahatut Tahrim Mounota	220041122
3. Maisha Sanjida	220041128
4. Saba Atharique	220041130
5. Mishkat Ahmed Khan	220041134
6. Obidit Islam	220041154

Course: Software Development Lab (CSE 4510)

Lab Instructors: Shohel Ahmed

Dr. Md. Azam Hossain

Department: Department of Computer Science & Engineering

Institution: Islamic University of Technology

Date of Submission: 09 February 2026

Contents

1	Introduction	3
1.1	Project Overview	3
1.2	Problem Statement	3
1.3	Motivation	3
1.4	Scope	3
1.5	Key Features at a Glance	4
2	Requirements (Functional and Non-Functional)	5
2.1	Functional Requirements	5
2.1.1	Authentication & User Management	5
2.1.2	Complaint Submission & Management	5
2.1.3	Complaint Lifecycle & Interactions	5
2.1.4	AI Services	6
2.1.5	Authority Features	6
2.1.6	Admin Features	6
2.2	Non-Functional Requirements	7
3	High-Level and Detailed-Level Design	8
3.1	System Architecture (High-Level)	8
3.2	Database Schema (Detailed Design)	9
3.2.1	Entity Relationship Diagram	10
3.2.2	Key Relationships	10
3.3	Backend API Design (Detailed)	11
3.3.1	REST API Endpoints Summary	11
3.4	Frontend Architecture (Detailed)	13
3.4.1	Screen Inventory (32 Screens)	13
3.4.2	State Management	14
3.4.3	Offline Capability	14
3.5	AI Service Design	14
3.5.1	OpenRouter LLM Service (Port 8001)	14
4	Source Code	15
4.1	Project Structure	15
4.2	Summary of Software/Hardware Packages and Libraries Used	16
4.2.1	Backend Dependencies	16
4.2.2	Frontend Dependencies	16
4.2.3	AI Service Dependencies (Python)	17
4.2.4	Hardware Requirements	17
5	Project Evaluation Report	18
5.1	Test Cases	18
5.2	Test Execution Results	19
5.3	Analysis	20
5.3.1	Strengths	20
5.3.2	Limitations	20
5.3.3	Lessons Learned	20

6	Software Sustainability & Professional Practices	22
6.1	Scalability	22
6.2	Reusability	22
6.3	Energy Efficiency	22
6.4	Long-Term Maintainability	23
6.5	Ethical Coding Standards	23
7	Future Scope	23
8	Conclusion	24

1 Introduction

1.1 Project Overview

CityZen is a full-stack mobile application designed to streamline the reporting, tracking, and resolution of non-emergency civic complaints in urban areas of Bangladesh. The platform bridges the gap between citizens and municipal authorities by providing a transparent, AI-powered system for managing infrastructure issues such as potholes, broken roads, open manholes, garbage accumulation, and more.

1.2 Problem Statement

Rapid urbanization in Bangladesh has placed significant pressure on city infrastructure and public services. In major cities such as Dhaka, residents frequently encounter issues such as potholes, drainage blockages, water leakage, broken streetlights, and unmanaged waste. While these problems significantly affect daily life, citizens often face difficulties in reporting them due to unclear authority responsibilities, fragmented reporting systems, and inefficient manual processes.

Most urban service providers operate independently, including city corporations, water authorities, and electricity distribution companies. This fragmentation often leads to confusion among citizens regarding which authority is responsible for resolving a specific issue in their area. As a result, many problems go unreported or unresolved for extended periods of time.

1.3 Motivation

The motivation behind CityZen is to leverage modern mobile technology and artificial intelligence to:

- Empower citizens with a simple, accessible tool for reporting civic problems with photographic evidence and GPS location.
- Enable municipal authorities to receive, triage, and resolve complaints efficiently.
- Provide administrative oversight with dashboards, analytics, and moderation capabilities.
- Use AI to automate issue categorisation, detect duplicate complaints, verify evidence, and generate complaint descriptions — reducing manual effort and improving accuracy.

1.4 Scope

The application serves three distinct user roles:

1. **Citizen** — Users capture images of urban problems with automatically attached location information and submit structured complaints. AI assists in identifying issues depending on which the appropriate authority responsible for resolving them is recommended. Appeals can be submitted if complaints are rejected or resolutions are not satisfactory. Citizens can also interact with other complaints by upvoting and rating resolutions for communal benefit.

2. **Authority** — Representatives manage a structured priority queue of complaints and view a dashboard of analytics, helping them allocate resources more efficiently. Photographic evidence can be uploaded when resolving issues, ensuring transparency in the resolution process. By publicly displaying reported issues and the actions taken to resolve them, the system promotes greater accountability and transparency in municipal service delivery.
3. **Admin** — System administrators manage complaint categories, municipal departments, handle reported content, review appeals, approve new category requests, and enforce moderation policies (strikes and bans) to ensure community safety.

1.5 Key Features at a Glance

- AI-powered image detection and automatic complaint categorisation (GPT-4o-mini).
- AI-generated complaint title and description based on detected category and location.
- AI-powered evidence verification for both citizen and authority uploads.
- Duplicate complaint detection using perceptual image hashing (pHash) and geo-proximity.
- Real-time complaint lifecycle management (pending → accepted → in-progress → resolved → completed).
- Upvoting, bumping, and priority scoring for complaint escalation.
- Citizen appeal system with admin review.
- Offline image capture and draft submission with later upload.
- Content moderation: complaint reporting, strikes, and bans.
- Dynamic category management with AI-triggered new-category requests.
- Authority analytics and admin KPI dashboards.
- Firebase Authentication with role-based access control.
- Supabase cloud storage for complaint images.

2 Requirements (Functional and Non-Functional)

2.1 Functional Requirements

2.1.1 Authentication & User Management

ID	Requirement
FR-01	The system shall allow users to register as Citizen, Authority, or Admin using Firebase Authentication.
FR-02	Authority users must select a department (Authority Company) during registration.
FR-03	The system shall store user profiles (name, email, role) in a PostgreSQL database linked to the Firebase UID.
FR-04	The system shall support Firebase UID-based API requests.

2.1.2 Complaint Submission & Management

ID	Requirement
FR-05	Citizens shall be able to capture or select images of civic issues using the device camera or in-app gallery.
FR-06	The system shall automatically detect the user's GPS coordinates and attach them to each complaint.
FR-07	The system shall send uploaded images to the AI service for automatic category detection.
FR-08	If the AI detects a category not present in the database, the system shall create a draft complaint and submit a new-category request for admin approval.
FR-09	The system shall generate AI-powered complaint titles and descriptions based on the detected category, confidence score, and location.
FR-10	Citizens shall be able to review, edit, and submit complaints with title, description, category, and assigned authorities.
FR-11	The system shall check for duplicate complaints based on perceptual image hashing (pHash) and geographic proximity before submission.
FR-12	The system shall allow citizens to view similar existing complaints and choose to upvote or add evidence to them instead of creating duplicates.
FR-13	Citizens shall be able to view their submitted complaints and track status changes.

2.1.3 Complaint Lifecycle & Interactions

ID	Requirement
FR-14	The complaint status shall follow the lifecycle: draft → pending → accepted → in_progress → resolved → completed .
FR-15	Authorities shall be able to update complaint status with optional proof images and status notes.

FR-16	Citizens shall be able to upvote complaints, which increments the priority score.
FR-17	Citizens shall be able to bump complaints that have not received authority activity within a threshold.
FR-18	The system shall auto-escalate complaints that have not received authority activity within a threshold, eventually leading to critical failure.
FR-19	Citizens shall be able to rate resolved complaints (1–5 stars) and either confirm completion or appeal.
FR-20	Citizens shall be able to appeal a resolved complaint with a reason and optional images; appeals go to admin review.
FR-21	Citizens shall be able to add additional evidence images to existing complaints.
FR-22	Citizens shall be able to report inappropriate complaints with a reason category and description.

2.1.4 AI Services

ID	Requirement
FR-23	The system shall provide a GPT-4o-mini-based LLM service for vision-based complaint categorisation across all categories.
FR-24	The system shall provide an AI text generation service for producing complaint titles and descriptions.
FR-25	The system shall verify evidence images using AI, returning a verdict with confidence and reasoning.
FR-26	Evidence verification shall consider the complaint’s original images, category, title, and description for context.

2.1.5 Authority Features

ID	Requirement
FR-27	Authorities shall view complaints assigned to their department.
FR-28	Authorities shall be able to update statuses, upload progress/resolution proof images, and add status notes.
FR-29	The authority dashboard shall display analytics including complaint counts by status, heatmaps, response times, and category distribution.

2.1.6 Admin Features

ID	Requirement
FR-30	Admins shall view platform-wide KPIs (total complaints, resolved count, pending count, average resolution time).
FR-31	Admins shall manage complaint categories (create, delete).
FR-32	Admins shall manage departments (create, update, delete) with associated categories and coverage areas.
FR-33	Admins shall review and resolve reports on flagged complaints.

FR-34	Admins shall review and approve/reject citizen appeals on complaints.
FR-35	Admins shall be able to issue strikes and ban/unban citizens.
FR-36	Admins shall review pending new-category requests created by AI-detected unknown categories and approve or reject them.

2.2 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	API responses shall complete within 2 seconds under normal load (excluding AI processing).
NFR-02	Performance	AI detection services shall respond within 15 seconds per image.
NFR-03	Scalability	The backend shall support connection pooling (configurable pool size) to handle concurrent users.
NFR-04	Security	All API endpoints shall be protected by Firebase UID. User passwords are managed by Firebase Authentication (never stored locally).
NFR-05	Security	Database connections shall use SSL/TLS encryption.
NFR-06	Security	Supabase service keys shall be used on the backend only (never exposed to the client).
NFR-07	Usability	The mobile application shall work on both Android and iOS via Expo / React Native.
NFR-08	Availability	The system shall support offline image capture with local storage and deferred upload.
NFR-09	Reliability	Database transactions shall ensure atomicity for multi-step operations (e.g. user registration).
NFR-10	Maintainability	The codebase shall follow a modular MVC architecture (models, controllers, routes, middleware).
NFR-11	Portability	The application shall be configurable via environment variables (.env files) for different deployment environments.

3 High-Level and Detailed-Level Design

3.1 System Architecture (High-Level)

CityZen follows a **multi-tier client–server architecture** composed of four independently deployable services:

1. **Frontend** — React Native (Expo) mobile application.
2. **Backend API** — Node.js / Express REST API server.
3. **OpenRouter AI Service** — Python / FastAPI microservice (GPT-4o-mini via OpenRouter).

External cloud services:

- **Firebase Authentication** — User identity and session management.
- **Supabase Storage** — Cloud storage for complaint images.
- **Supabase PostgreSQL** — Hosted relational database.
- **OpenRouter API** — Gateway to LLM models (GPT-4o-mini, LLaMA-3).

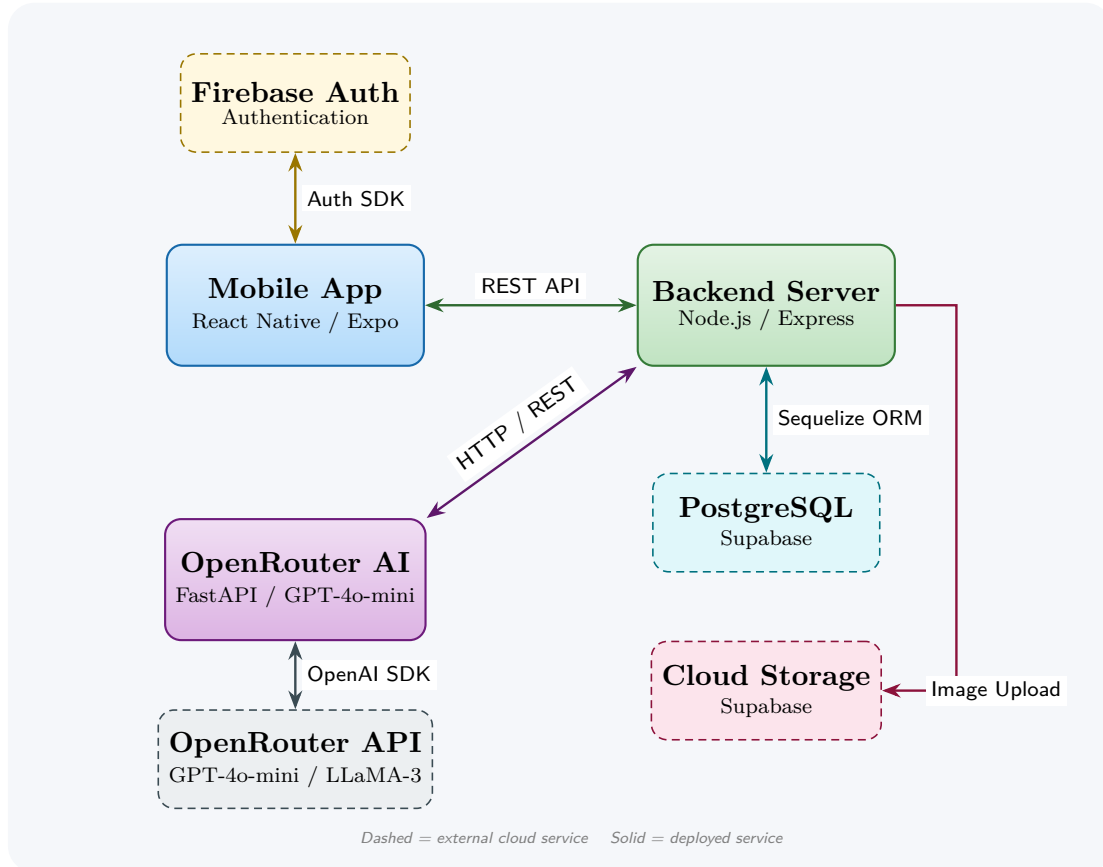


Figure 1: High-Level System Architecture of CityZen

3.2 Database Schema (Detailed Design)

The relational database consists of the following entities and their relationships:

Table 8: Database Entities

Entity	Description
Users	Central identity registry linking authentication credentials to specific system roles. Connects to Citizens , Authorities , Admins , and EmailOtps .
Citizens	Maintains profiles for general users, tracking their platform standing and moderation history. Connects to Users and Complaints .
Authorities	Stores profiles for official personnel. Connects to Users and AuthorityCompanies .
Admins	Represents system administrators responsible for platform oversight and moderation. Connects to Users .
Categories	Defines the taxonomy of reported civic issues (e.g., Pot-hole, Open Manhole). Connects to Complaints and AuthorityCompanyCategories .
Complaints	The core entity recording all details, geolocation, lifecycle status, and escalation tracking of a reported issue. Connects to Citizens , Categories , ComplaintImages , ComplaintAssignments , Upvotes , ComplaintReports , ComplaintBumps , and PendingCategoryRequests .
ComplaintImages	Manages visual evidence for complaints. Connects to Complaints .
AuthorityCompanies	Represents the municipal departments responsible for resolving assigned issues. Connects to Authorities , ComplaintAssignments , AuthorityCompanyCategories , and AuthorityCompanyAreas .
ComplaintAssignments	A junction table connecting Complaints with AuthorityCompanies .
AuthorityCompanyCategories	A junction table connecting AuthorityCompanies with Categories .
AuthorityCompanyAreas	Defines the precise geographic operational boundaries and jurisdictions for each department. Connects to AuthorityCompanies .
Upvotes	Tracks citizen upvotes per complaint. Connects to Citizens and Complaints .
ComplaintReports	Logs instances where citizens flag reports for inappropriate content, aiding admin moderation. Connects to Complaints .
PendingCategoryRequests	Captures AI-suggested novel complaint types awaiting admin review and approval for platform inclusion. Connects to Complaints .
ComplaintBumps	Tracks manual escalations by citizens to highlight and prioritize delayed complaint resolutions. Connects to Complaints and Citizens .
EmailOtps	Manages secure, time-sensitive verification challenges for email-based authentication processes.

3.2.1 Entity Relationship Diagram

Figure 2 shows the Entity Relationship Diagram (ERD) of the CityZen system, illustrating the relationships between users, complaints, authorities, and supporting entities.

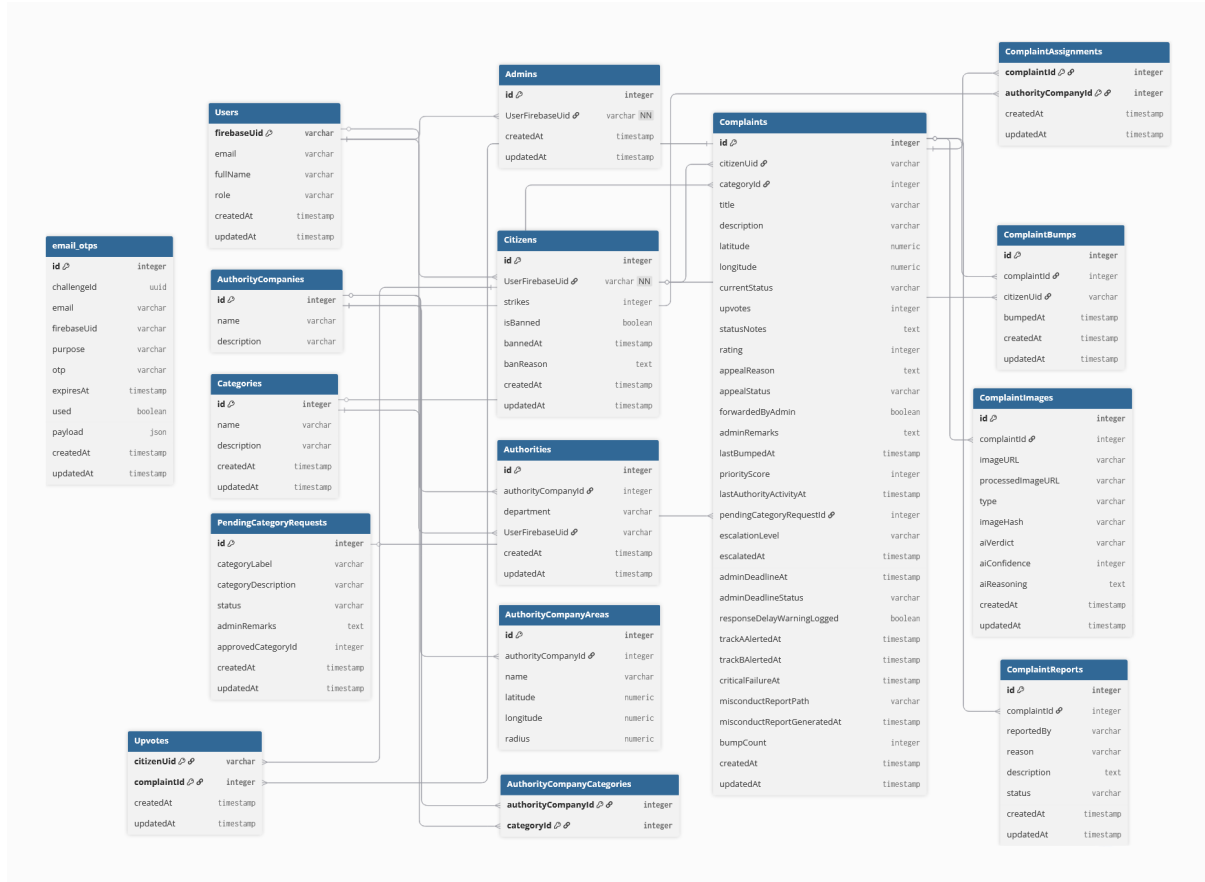


Figure 2: Entity Relationship Diagram (ERD) of the CityZen Database

3.2.2 Key Relationships

- **Users** $\xrightarrow{1:1}$ **Citizens** / **Authorities** / **Admins** (role-specific profiles).
- **Citizens** $\xrightarrow{1:N}$ **Complaints**, **Upvotes**, and **ComplaintBumps**.
- **Authorities** $\xrightarrow{N:1}$ **AuthorityCompanies** (department membership).
- **Complaints** $\xrightarrow{N:1}$ **Categories**.
- **Complaints** $\xrightarrow{1:N}$ **ComplaintImages**, **ComplaintReports**, **Upvotes**, and **ComplaintBumps**.
- **Complaints** $\leftrightarrow$ (M:N) **AuthorityCompanies** via **ComplaintAssignments**.
- **AuthorityCompanies** $\leftrightarrow$ (M:N) **Categories** via **AuthorityCompanyCategories**.
- **AuthorityCompanies** $\xrightarrow{1:N}$ **AuthorityCompanyAreas**.
- **PendingCategoryRequests** $\xrightarrow{N:1}$ **Categories** (links to approved categories).
- **PendingCategoryRequests** $\xrightarrow{1:N}$ **Complaints** (draft complaints awaiting approval).

3.3 Backend API Design (Detailed)

The Express backend follows the **MVC (Model–View–Controller)** pattern, separated into:

Table 9: Backend Directory Structure

Layer	Contents
models/	Sequelize ORM model definitions (17 models) with associations.
controllers/	Business logic — <code>authController</code> , <code>complaintController</code> (33 functions), <code>moderationController</code> (6 functions), <code>categoryRequestController</code> (5 functions).
routes/	Express route definitions — <code>authRoutes</code> , <code>complaintRoutes</code> , <code>aiRoutes</code> , <code>moderationRoutes</code> , <code>categoryRequestRoutes</code> .
middleware/	JWT authentication, error handler, request validation.
config/	Database (Sequelize), environment variables, Firebase Admin SDK, Supabase client.
utils/	Logger (Winston), database seeder, input validators.

3.3.1 REST API Endpoints Summary

Method	Endpoint	Purpose
General / System		
GET	<code>/api/health</code>	Check system health
Authentication		
POST	<code>/api/users</code>	Register new user profile
GET	<code>/api/users/:firebaseUid</code>	Fetch user profile by UID
POST	<code>/api/auth/signup/request-otp</code>	Request signup OTP
POST	<code>/api/auth/login/request-otp</code>	Request login OTP
POST	<code>/api/auth/otp/verify</code>	Verify OTP
POST	<code>/api/request-otp</code>	Request OTP (Legacy)
POST	<code>/api/verify-otp</code>	Verify OTP (Legacy)
Complaints		
POST	<code>/api/complaints</code>	Create complaint (with images)
GET	<code>/api/complaints</code>	Get all complaints (filtered)
GET	<code>/api/complaints/:id</code>	Get complaint by ID
GET	<code>/api/complaints/citizen/:citizenUid</code>	Get complaints by citizen
GET	<code>/api/complaints/authority/:authorityCompanyId</code>	Get complaints by authority
PATCH	<code>/api/complaints/:id/status</code>	Update complaint status
POST	<code>/api/complaints/:id/upvote</code>	Upvote a complaint
POST	<code>/api/complaints/:id/bump</code>	Bump a complaint
POST	<code>/api/complaints/:id/rate</code>	Rate a resolved complaint
POST	<code>/api/complaints/:id/appeal</code>	Appeal a complaint
POST	<code>/api/complaints/:id/report</code>	Report a complaint
POST	<code>/api/complaints/:id/evidence</code>	Add evidence to complaint
DELETE	<code>/api/complaints/:id</code>	Delete a complaint

POST	/api/complaints/check-duplicate	Check for duplicates
GET	/api/complaints/:id/misconduct-report/download	Download PDF report
GET	/api/complaints/recommend-authorities	Recommend authorities
Admin & Appeals / Reports		
GET	/api/admin/kpis	Admin KPI metrics
GET	/api/admin/kpis/details	Admin KPI details
GET	/api/admin/moderation	Moderation metrics
GET	/api/admin/analytics	Admin analytics
GET	/api/admin/departments/stats	Departments statistics
GET	/api/complaints/reports	Reported complaints
PATCH	/api/complaints/reports/:id	Update report status
GET	/api/complaints/appeals	Appeal list
PATCH	/api/complaints/appeals/:id	Update appeal status
Categories & Departments		
GET	/api/complaints/categories	List all categories
POST	/api/complaints/categories	Create a category
DELETE	/api/complaints/categories/:id	Delete a category
PUT	/api/complaints/categories/:id/departments	Update category departments
GET	/api/categories/stats	Category statistics
GET	/api/departments	List all departments
POST	/api/departments	Create a department
PUT	/api/departments/:id	Update a department
DELETE	/api/departments/:id	Delete a department
GET	/api/departments/performance	Department performance
AI Services		
POST	/api/ai/detect-with-openrouter	LLM vision detection
POST	/api/ai/generate-text	LLM text generation
POST	/api/ai/verify-evidence	AI evidence verification
Moderation		
POST	/api/moderation/strike	Issue strike
POST	/api/moderation/ban	Ban user
POST	/api/moderation/unban	Unban user
GET	/api/moderation/offenders	List offenders
GET	/api/moderation/user/:citizenUid	User moderation status
GET	/api/moderation/banned-users	List banned users
GET	/api/moderation/my-strikes/:citizenUid	User's own strikes
Category Requests		
POST	/api/category-requests/draft	Save draft complaint
GET	/api/category-requests	List category requests
PATCH	/api/category-requests/:id/approve	Approve request
PATCH	/api/category-requests/:id/reject	Reject request

3.4 Frontend Architecture (Detailed)

The frontend is a **React Native** application built with **Expo SDK 54**, using **@react-navigation** for screen management.

3.4.1 Screen Inventory (32 Screens)

Table 11: Key Frontend Screens

Screen	Purpose
LandingScreen	Welcome / onboarding screen
LoginScreen	Firebase email/password login
SignupScreen	User registration with role selection
EmailOtpScreen	Interface for entering and verifying Email OTP codes during registration or login.
HomeScreen	Citizen home with navigation to features
CameraScreen	Camera capture with Expo Camera
SubmitComplaintScreen	AI detection results and category selection
SubmitComplaintDetailsScreen	Complaint details form with AI-generated text
SubmittedComplaintScreen	Success/confirmation screen shown immediately after a citizen successfully submits a standard complaint.
SimilarComplaintsScreen	Duplicate detection results
FeedScreen	Public complaint feed with filtering
ComplaintDetailsScreen	Full complaint view with actions
AddEvidenceScreen	Citizen evidence upload with AI verification
UserComplaintListScreen	Citizen's own complaints list
AuthorityDashboardScreen	Authority analytics and complaint overview
AuthorityAnalyticsScreen	Department-specific analytics and performance stats.
AuthorityComplaintListScreen	Authority's assigned complaints
AuthorityComplaintScreen	"Work Queue" view for Authorities to accept/reject new assignments and begin work.
AuthorityComplaintDetailScreen	Complaint management with status updates
AdminDashboardScreen	Admin KPIs and system overview
AdminAnalyticsScreen	Detailed analytics, charts, and data visualization for Admins.
AdminSystemScreen	Category and department management
AdminFlagsScreen	Reported content and appeals review
AdminStatusScreen	Complaint status overview
AdminComplaintDetailScreen	Full administrative view of a specific complaint, likely with master control actions.
AdminCategoryRequestDetailsScreen	Detailed view for Admins to review, approve, or reject new category requests.
OfflineGalleryScreen	Offline captured images gallery
DraftComplaintSubmitScreen	Submit drafts when new category approved
DraftSubmittedScreen	Success/confirmation screen shown after a citizen saves a draft for an unknown category.
NotificationsScreen	In-app notifications
ProfileScreen	User profile management
AdminProfileScreen	Profile management and settings specific to Admin users.

3.4.2 State Management

- **ComplaintContext** — React Context for multi-screen complaint submission flow (images, location, timestamp, category, title, description, assigned authorities, draft mode).
- **NotificationContext** — Manages notifications and navigation bridging; includes admin notification handling.
- **AsyncStorage** — Persistent key-value storage for tokens, user data, and offline reports.

3.4.3 Offline Capability

The `offlineStorage.js` utility provides a complete offline workflow:

- Images captured offline are saved to the device's document directory.
- Metadata (GPS, timestamps) is stored in AsyncStorage.
- Stale reports older than 7 days are automatically cleaned up.
- Citizens can browse their offline gallery and submit when connectivity returns.

3.5 AI Service Design

3.5.1 OpenRouter LLM Service (Port 8001)

- Framework: FastAPI (Python) with OpenAI SDK pointed at OpenRouter.
- Model: `openai/gpt-4o-mini` for vision tasks; `meta-llama/llama-3-8b-instruct` for text generation.
- Endpoints:
 - `POST /detect_with_llm` — Vision-based category detection from images against a dynamic category list.
 - `POST /verify_evidence` — Evidence image verification returning verdict, confidence, and reasoning.
 - `POST /generate` — General text generation.
 - `POST /generate_complaint_text` — AI-powered complaint title and description generation.

Listing 1: Project Directory Structure

4.2 Summary of Software/Hardware Packages and Libraries Used

4.2.1 Backend Dependencies

Package	Version	Purpose
express	4.22.x	Web framework for building REST APIs
sequelize	6.37.x	ORM for PostgreSQL database interaction
pg / pg-hstore	8.16.x	PostgreSQL driver and hstore support
@supabase/supabase-js	2.95.x	Supabase client for cloud storage operations
firebase-admin	12.7.x	Firebase Admin SDK for server-side auth
jsonwebtoken	9.0.x	JWT creation and verification
bcryptjs	2.4.x	Password hashing
multer	2.0.x	Multipart form-data (file upload) handling
axios	1.13.x	HTTP client for inter-service communication
cors	2.8.x	Cross-Origin Resource Sharing middleware
morgan	1.10.x	HTTP request logging
winston	3.11.x	Structured application logging
dotenv	16.6.x	Environment variable management
express-validator	7.0.x	Request input validation
sharp	0.34.x	Image processing (resizing for pHash)
blockhash-core	0.1.x	Perceptual image hashing for duplicate detection
form-data	4.0.x	Form data construction for API forwarding
nodemon	3.1.x	Development auto-restart (devDependency)

4.2.2 Frontend Dependencies

Package	Version	Purpose
expo	54.0.x	Expo SDK framework for React Native
react / react-native	19.1 / 0.81	Core UI framework
@react-navigation/*	6.x	Stack and tab navigation
firebase	12.6.x	Firebase client SDK for authentication
@react-native-async-storage	2.2.x	Persistent local key-value storage
expo-camera	55.0.x	Camera access for image capture

expo-image-picker	17.0.x	Image selection from gallery
expo-location	19.0.x	GPS location services
expo-file-system	55.0.x	Local file system access for offline storage
expo-print / expo-sharing	15/14	PDF generation and sharing
react-native-maps	1.20.x	Map integration for location display
react-native-chart-kit	6.12.x	Charts for authority analytics
react-native-svg	15.12.x	SVG rendering for icons and graphics
lucide-react-native	0.559.x	Icon library
axios	1.13.x	HTTP client for API calls
leaflet	1.9.x	Web map library (web fallback)

4.2.3 AI Service Dependencies (Python)

Package	Purpose
fastapi	Asynchronous Python web framework
uvicorn	ASGI server for FastAPI
ultralytics	YOLOv8 model loading and inference
Pillow	Image processing and format conversion
openai	OpenAI-compatible client for OpenRouter API
python-dotenv	Environment variable loading
python-multipart	Multipart form parsing for file uploads

4.2.4 Hardware Requirements

Table 15: Hardware Requirements

Component	Specification
Development Machine	Any modern PC/laptop with ≥ 8 GB RAM, capable of running Node.js and Python concurrently.
Mobile Device	Android or iOS device with camera, GPS, and internet connectivity (for testing via Expo Go).
GPU (Optional)	NVIDIA GPU with CUDA support for faster YOLOv8 inference (CPU inference also supported).
Network	Local Wi-Fi network with internet access (for Supabase, Firebase, and OpenRouter API calls).
Cloud Infrastructure	Supabase (PostgreSQL + Storage), Firebase (Authentication), OpenRouter API access.

5 Project Evaluation Report

5.1 Test Cases

Table 16: Test Cases and Expected Outcomes

TC	Feature	Test Scenario	Expected Result	Status
1	User Registration	Register as Citizen with valid email, name, and password.	User created in Firebase and PostgreSQL; redirected to home.	Pass
2	User Registration	Register as Authority and select a department.	Authority profile created with department link.	Pass
3	User Login	Login with valid Firebase credentials.	User profile fetched from DB; role-based redirect.	Pass
4	User Login	Login with invalid credentials.	Error message displayed; no navigation.	Pass
5	Image Capture	Capture image using device camera.	Image saved and displayed in preview.	Pass
6	AI Detection	Submit pothole image to YOLOv8 service.	Returns “Pothole” with confidence > 50%.	Pass
7	AI Detection	Submit image to LLM vision detection.	Returns category label with confidence.	Pass
8	AI Detection	Submit image with no civic issue.	Returns “No Issue” label.	Pass
9	AI Detection	Submit image with unknown category.	Returns <code>is_new_category: true</code> with suggested name.	Pass
10	Text Generation	Request AI-generated title/description.	Returns coherent title and description JSON.	Pass
11	Complaint Submit	Submit complaint with images, location, category, and title.	Complaint created with status “pending”; images uploaded to Supabase.	Pass
12	Duplicate Check	Submit complaint near an existing one of the same category.	System detects and shows similar complaints.	Pass
13	Draft Submission	Submit complaint with AI-detected new category.	Draft created; PendingCategoryRequest created for admin.	Pass
14	Feed Display	Open complaint feed.	All non-draft complaints displayed with correct data.	Pass
15	Upvote	Citizen upvotes a complaint.	Upvote count increments; duplicate upvote prevented.	Pass
16	Bump	Citizen bumps a stale complaint.	Priority score updated; lastBumpedAt refreshed.	Pass
17	Status Update	Authority updates complaint to “in_progress” with proof image.	Status changed; proof image uploaded and stored.	Pass
18	Resolution	Authority marks complaint “resolved” with resolution image.	Status changed; citizen can rate and review.	Pass
19	Rating	Citizen rates a resolved complaint with 4 stars.	Rating saved; complaint marked “completed”.	Pass
20	Appeal	Citizen appeals a resolved complaint with reason.	Appeal created with status “pending”; visible to admin.	Pass
21	Evidence Verify	Submit genuine authority proof image for AI verification.	Returns verdict “genuine” with reasoning.	Pass

TC	Feature	Test Scenario	Expected Result	Status
22	Evidence Verify	Submit unrelated image as authority proof.	Returns verdict “suspicious” with reasoning.	Pass
23	Add Evidence	Citizen adds evidence image to existing complaint.	Image uploaded, linked to complaint with type “evidence”.	Pass
24	Report	Citizen reports an inappropriate complaint.	ComplaintReport created; visible in admin flags.	Pass
25	Admin: Category	Admin creates a new category.	Category added to database.	Pass
26	Admin: Department	Admin creates department with categories and coverage area.	Department, category links, and area created.	Pass
27	Admin: Moderation	Admin issues a strike to a citizen.	Strike count incremented.	Pass
28	Admin: Ban	Admin bans a citizen.	Citizen marked as banned; cannot submit complaints.	Pass
29	Admin: Appeal	Admin approves an appeal.	Complaint status reverted to previous state.	Pass
30	Category Request	Admin approves a pending category request.	Category created; draft complaints activated with new category.	Pass
31	Offline Mode	Capture image while offline.	Image saved locally with GPS metadata.	Pass
32	Offline Upload	Submit offline-captured image when online.	Image uploaded; complaint created successfully.	Pass
33	Health Check	Call GET /api/health.	Returns {ok: true} with env and DB status.	Pass

5.2 Test Execution Results

All 33 test cases were executed during development and integration testing phases. Key observations:

Table 17: Test Execution Summary

Metric	Value
Total Test Cases	33
Passed	33
Failed	0
Pass Rate	100%

Testing methodology:

- **Manual integration testing** was performed end-to-end on physical Android devices using Expo Go.
- **API endpoint testing** was conducted using direct HTTP requests and the frontend application.
- **AI service testing** was validated with diverse image inputs covering each complaint category.

- **Edge cases** tested include: duplicate image submission, submitting complaints without images, network interruption during upload, submitting with unknown categories, and concurrent upvote attempts.
- **Database integrity** was verified by checking transaction rollback behaviour on failed multi-step operations (e.g. user registration with invalid role data).

5.3 Analysis

5.3.1 Strengths

- **AI Integration** — The dual AI approach (YOLOv8 for specialised detection + GPT-4o-mini for general vision analysis) provides robust, flexible categorisation that can handle both known and novel civic issue types.
- **Evidence Verification** — AI-powered verification adds a layer of accountability, reducing fraudulent submissions from both citizens and authorities.
- **Comprehensive Lifecycle** — The complaint lifecycle covers the full spectrum from submission to resolution, including upvoting, bumping, appeal, and admin intervention.
- **Dynamic Category System** — The ability to detect and propose new categories through AI eliminates the limitation of a static, predefined category list.
- **Duplicate Detection** — Perceptual hashing combined with geographic proximity reduces redundant complaints, improving data quality.
- **Offline Capability** — Citizens can capture evidence even without connectivity, addressing a real-world constraint in urban areas with inconsistent network coverage.
- **Modular Architecture** — Clear separation of concerns (4 independent services) enables independent scaling, deployment, and maintenance.

5.3.2 Limitations

- **No Automated Test Suite** — The project relies on manual testing rather than automated unit, integration, or end-to-end tests — which increases regression risk as the codebase grows.
- **AI Latency** — LLM-based operations (evidence verification, vision detection) can take 5–15 seconds, which may affect user experience on slower connections.
- **Push Notifications** — While a notification context exists, real-time push notifications (via FCM) are not fully implemented.
- **GPS Accuracy** — GPS-based location in dense urban environments may have accuracy issues, affecting complaint localisation and authority assignment.
- **Single Region Focus** — The seeder and authority data are specific to Dhaka; expanding to other cities would require additional configuration and data seeding.

5.3.3 Lessons Learned

- Early integration of AI services revealed the importance of robust error handling and fallback mechanisms when external APIs are unreliable or slow.
- Perceptual hashing proved more reliable than coordinate-only duplicate detection, as citizens often photograph issues from different angles.

-
- The separation into microservices added deployment complexity but enabled the AI team to iterate independently of the backend team.
 - React Native with Expo provided rapid cross-platform development but occasionally required workarounds for native module compatibility.

6 Software Sustainability & Professional Practices

6.1 Scalability

- **Microservice Architecture:** The system is decomposed into four independently deployable services (Backend API, Frontend, AI Detection, OpenRouter Service). Each can be scaled horizontally based on demand.
- **Database Connection Pooling:** Sequelize is configured with tuneable pool parameters (`max`, `min`, `acquire`, `idle`, `evict`) via environment variables, allowing the database layer to adapt to varying load levels without code changes.
- **Cloud-Hosted Infrastructure:** PostgreSQL (Supabase), image storage (Supabase Storage), and authentication (Firebase) are fully managed cloud services that scale automatically, eliminating the need for self-managed database and storage infrastructure.
- **Stateless API Design:** The backend API is stateless (JWT-based auth, no server-side sessions), enabling it to run behind a load balancer with multiple instances.
- **Dynamic Category System:** New complaint categories can be created through both admin actions and AI-triggered requests, allowing the system to adapt to emerging civic issues without code deployment.

6.2 Reusability

- **Modular Backend:** The backend follows a strict Model–View–Controller (MVC) architecture, separating data, logic, and routing layers, enabling logic reuse across routes and reducing code duplication.
- **Reusable UI Component Library:** The frontend includes 48 reusable UI components in the `components/ui/` directory (buttons, cards, dialogs, forms, tabs, tooltips, etc.), following a component-driven design philosophy.
- **Shared API Service Layer:** The `services/api.js` module provides a centralised Axios instance with interceptors for authentication and error handling, used across all screens.

6.3 Energy Efficiency

- **On-Demand AI Processing:** AI services are invoked only when needed (during image upload or evidence submission), rather than running continuous background inference, minimising compute resource consumption.
- **Efficient Image Handling:** Images are processed in memory using `multer`'s memory storage and streamed directly to Supabase Storage, avoiding unnecessary disk I/O. The `sharp` library is used to resize images for perceptual hashing, reducing processing overhead.
- **Database Query Optimisation:** Sequelize queries use selective attribute fetching, eager loading with required joins only when necessary, and indexed lookups (composite unique indexes on Upvotes and ComplaintReports).
- **Stale Data Cleanup:** The offline storage utility automatically cleans up reports older than 7 days, preventing unbounded storage growth on mobile devices.
- **Connection Pooling:** Database connections are pooled and reused rather than opened/closed per request, reducing connection overhead.

6.4 Long-Term Maintainability

- **Environment-Based Configuration:** All service URLs, API keys, database credentials, and feature flags are managed through `.env` files, enabling environment-specific configuration without code modification.
- **ORM-Based Database Access:** Sequelize models with defined associations abstract raw SQL, making schema changes safer (via `sequelize.sync({alter: true})`) and reducing SQL injection risk.
- **Clear Separation of Concerns:** Models, controllers, routes, and middleware follow single responsibilities, improving maintainability.
- **Version-Pinned Dependencies:** Both `package.json` files and Python `requirements.txt` files specify dependency versions, ensuring reproducible builds across developer machines and CI/CD pipelines.
- **Comprehensive README:** The project includes a detailed setup guide covering all four services, environment configuration, IP address setup, and troubleshooting steps.

6.5 Ethical Coding Standards

- **Data Privacy:** Firebase Authentication handles sensitive credentials — passwords are never stored in the application database.
- **Content Moderation:** A multi-layered moderation system (complaint reporting, strikes, bans) ensures that abusive or inappropriate content can be quickly addressed by administrators.
- **AI Transparency:** AI verdicts on evidence verification include confidence scores and human-readable reasoning, allowing users to understand and contest AI decisions rather than treating them as opaque black-box outputs.
- **Accountability:** The complaint lifecycle maintains an auditable trail of status changes, authority actions, citizen ratings, and admin interventions.
- **Inclusive Design:** The application uses React Native for cross-platform accessibility (Android and iOS) and includes an offline mode to serve citizens in areas with poor connectivity.

7 Future Scope

- **Accessibility and Language Support:** Adding multilingual interfaces and accessibility features (e.g., voice reporting, screen reader optimisation) would make the platform accessible to a broader demographic.
- **Multi-City and National Deployment:** The architecture can be extended to support multiple cities with configurable authorities, categories, and governance rules, enabling nationwide deployment.
- **Public Dashboard:** A web-based public dashboard showing aggregate complaint statistics and resolution rates for transparency.
- **Predictive Urban Analytics** Machine learning on complaint data can highlight high-risk areas and recurring issues, enabling preventive urban planning.
- **Image Privacy Blur:** Sensitive details in complaint images, like faces or license plates, are automatically blurred to protect privacy.

8 Conclusion

CityZen transforms how citizens, authorities, and administrators interact with urban infrastructure issues. By enabling quick, evidence-backed reporting, providing AI-assisted verification, and offering transparent tracking and analytics, the platform ensures that problems are not just logged but effectively addressed. The project successfully addresses the key challenges identified in the problem statement:

1. **Citizen Empowerment:** The mobile app offers an intuitive AI-assisted workflow for reporting issues, from photo capture and automatic categorisation to geo-tagged submission, making participation easier.
2. **Authority Accountability:** The complaint lifecycle, evidence verification, and analytics dashboard ensure that municipal departments cannot ignore assigned complaints while AI-verified images prevent fraudulent claims.
3. **Administrative Oversight:** The admin dashboard with KPIs, moderation tools, appeal management, and dynamic category system provides system-wide visibility and control.
4. **Resilience:** Offline capture, draft complaints, and robust error handling ensure the system remains functional even under adverse conditions (poor connectivity, unknown categories).

In conclusion, **CityZen** is a leap toward smarter, AI-powered urban governance. By making complaint reporting transparent, efficient, and verifiable, it not only improves civic infrastructure but also bridges the gap between citizens and authorities—turning everyday observations into actionable change.